

the currently displayed component. If the current component is the first, then the last is shown.



- `void show( Container, String )` - shows the component whose name matches the String passed in. If no components match the name, the method does nothing.

The `CardLayout` is instantiated with a horizontal gap of 10 pixels and a vertical gap of 5 pixels. Since only one component can be displayed at a time, the gaps specify a margin around the components, rather than between the components, as is the case with the `BorderLayout`.

Grid Layout

It lays out components in a grid, with each component stretched to fill its grid cell. Horizontal and vertical gaps between components can be specified. You should use it when you need to create something like a calendar or a spreadsheet.

You can set up the number of rows and columns it contains either at the time of instantiation, or anytime thereafter.

```
GridLayout gl = new GridLayout( 3, 0, 10, 10 );
```

GridBag Layout

It arranges components in a grid using an elaborate set of grid constraints. You should use it when you're building complicated input forms, such as a container that has components for name, address, city, state and zip.

Unlike `GridLayout`, where you explicitly specify the number of rows and columns in the grid, the `GridBagLayout` itself determines the number of rows and columns from constraints placed upon the components it lays out. Also, unlike `GridLayout`, `GridBagLayout` allows components to span more than one rigid cell. Components may also overlap.

The `GridBagLayout` is one of the toughest and most complex in all of AWT. Therefore, we will use divide and conquer to learn it. Lets see an example of this.